

8085 Microprocessor Programming Assignment

Jadavpur University - Electrical Engineering (2024)

1. Separate Even and Odd Numbers from an Array

Logic: We will load the count N , then iterate through the array. For each number, we will check the least significant bit (LSB) using the 'ANI 01H' instruction. If the result is 0, the number is even; if 1, it is odd. We maintain separate pointers for the even and odd destination arrays.

Memory Map Setup

Input Array Start: 2000H (Count N at 2000H, data from 2001H)
Even Array Start: 2100H (Count at 2100H, data from 2101H)
Odd Array Start: 2200H (Count at 2200H, data from 2201H)

```
1 LXI H, 2000H      ; HL points to source array (Count N)
2 MOV C, M          ; C = N (Loop counter)
3 LXI D, 2101H      ; DE points to even array data start
4 LXI B, 2201H      ; BC points to odd array data start
5 MVI D, 00H        ; D will act as Even Count (reusing D register is
    bad, let's use memory or push/pop. Let's use separate
    registers)
6
7 ; Corrected pointer initialization
8 LXI H, 2000H      ; Source pointer
9 MOV C, M          ; C = Count (N)
10 MVI D, 00H        ; D = Even numbers count
11 MVI E, 00H        ; E = Odd numbers count
12 INX H             ; Point to first data element
13
14 LOOP:
15     MOV A, M       ; Load number in accumulator
16     ANI 01H        ; Mask all bits except LSB
17     JNZ ODD        ; If LSB is 1, it's odd
18
19 EVEN:
20     MOV A, M       ; Reload the original number
21     PUSH H          ; Save source pointer
22     LXI H, 2101H    ; Base of Even array
23     MOV L, D        ; Add offset
24     ADD L
25     MOV L, A        ; Store address
26     ; A simpler way is to maintain separate 16-bit pointers.
```

Refined and optimized approach using proper 16-bit pointers:

```

1      LXI H, 2000H      ; HL = Source Array (Count N)
2      MOV C, M          ; C = N
3      LXI D, 2101H      ; DE = Even Array Destination
4      PUSH B            ; Save main counter (BC)
5      LXI B, 2201H      ; BC = Odd Array Destination
6      POP PSW           ; (Hack to get C back, better to use standard
                          ; regs)
7
8      ; Let's use Memory for pointers to avoid register exhaustion.
9      LXI H, 2000H
10     MOV C, M           ; C = Total count
11     INX H
12 NEXT:
13     MOV A, M           ; Load data
14     RRC                ; Check LSB in Carry Flag
15     JC IS_ODD
16
17 IS_EVEN:
18     RLC                ; Restore original number in A
19     PUSH H             ; Save source pointer
20     LXI H, (EVEN_PTR)  ; Load Even pointer from memory
21     MOV M, A           ; Store in Even array
22     INX H              ; Increment Even pointer
23     SHLD EVEN_PTR      ; Save updated pointer
24     POP H              ; Restore source pointer
25     JMP CONTINUE
26
27 IS_ODD:
28     RLC                ; Restore original number
29     PUSH H             ; Save source pointer
30     LXI H, (ODD_PTR)   ; Load Odd pointer from memory
31     MOV M, A           ; Store in Odd array
32     INX H              ; Increment Odd pointer
33     SHLD ODD_PTR       ; Save updated pointer
34     POP H              ; Restore source pointer
35
36 CONTINUE:
37     INX H
38     DCR C
39     JNZ NEXT
40     HLT

```

2. Sort Array in Ascending Order and Count Repeats

Logic: We will implement the Bubble Sort algorithm. After sorting, identical numbers will be adjacent to each other. We will then perform a single pass through the sorted array to count how many times an element is equal to the next element.

```

1      ; Part 1: Bubble Sort

```

```

2      LXI H, 2000H      ; Array count address
3      MOV C, M          ; C = N
4      DCR C            ; N-1 passes
5  OUTER:
6      MOV D, C          ; D = comparisons for this pass
7      LXI H, 2001H      ; Point to first element
8  INNER:
9      MOV A, M          ; Load A[i]
10     INX H
11     CMP M             ; Compare with A[i+1]
12     JC NO_SWAP        ; If A[i] < A[i+1], no swap
13     JZ NO_SWAP        ; If equal, no swap
14
15     ; Swap A[i] and A[i+1]
16     MOV B, M          ; B = A[i+1]
17     MOV M, A          ; A[i+1] = A[i]
18     DCX H             ; Point back to A[i]
19     MOV M, B          ; A[i] = B
20     INX H             ; Point to A[i+1] again
21  NO_SWAP:
22     DCR D
23     JNZ INNER
24     DCR C
25     JNZ OUTER
26
27  ; Part 2: Count Repeated Numbers
28     LXI H, 2000H
29     MOV C, M          ; Total elements N
30     DCR C            ; N-1 comparisons
31     MVI B, 00H        ; B = Duplicate counter
32     LXI H, 2001H      ; First element
33  COUNT_LOOP:
34     MOV A, M          ; A[i]
35     INX H
36     CMP M             ; Compare with A[i+1]
37     JNZ NOT_REP
38     INR B             ; Increment counter if equal
39  NOT_REP:
40     DCR C
41     JNZ COUNT_LOOP
42
43     ; Store the duplicate count at 2100H
44     LXI H, 2100H
45     MOV M, B
46     HLT

```

3. Factorial of an 8-bit Number with Flowchart

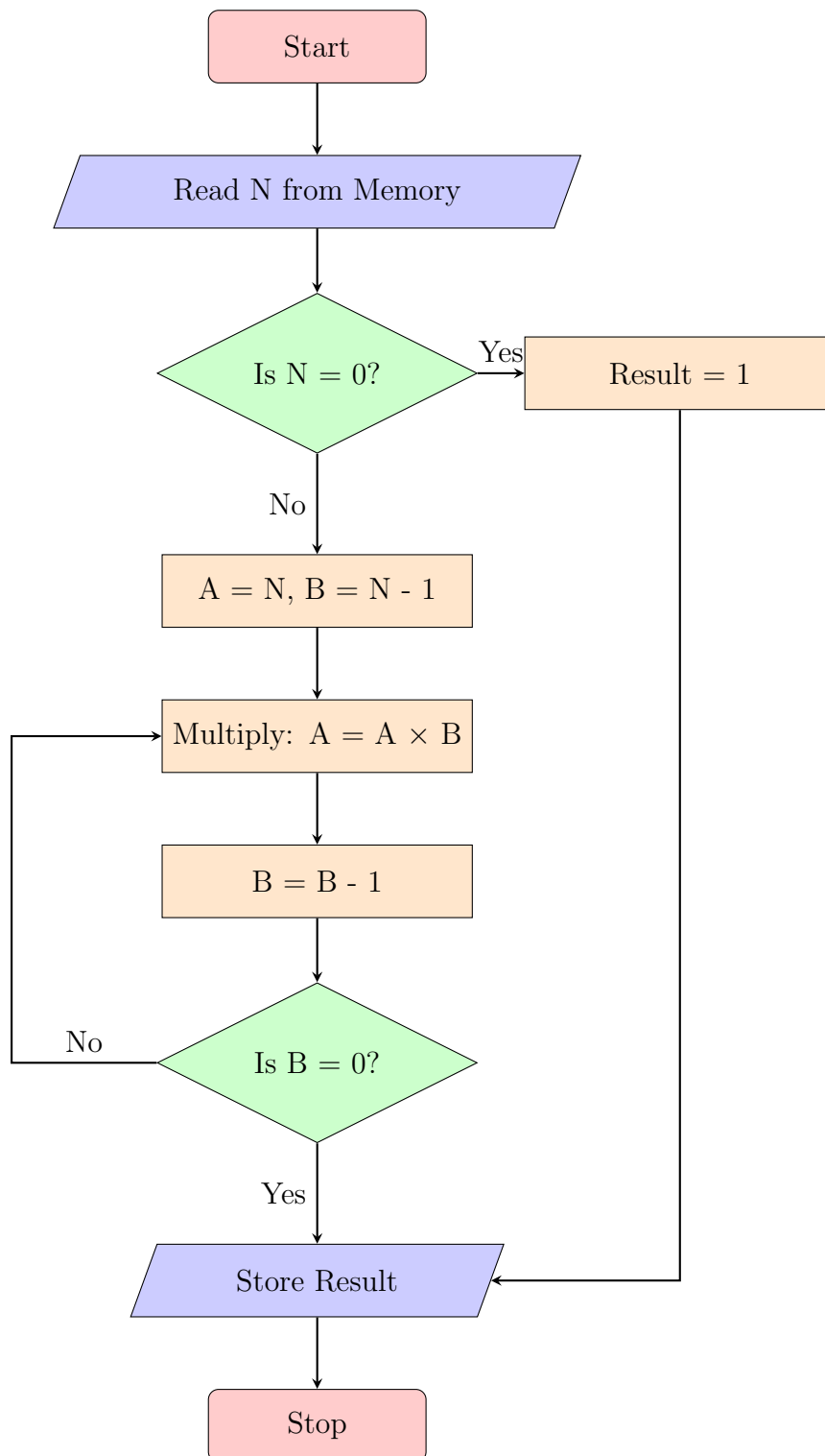
Logic: Factorial is found by repeated multiplication: $N! = N \times (N-1) \times (N-2) \times \dots \times 1$. We use a subroutine for multiplication since the 8085 does not have a native ‘MUL’

instruction.

Assembly Code

```
1      LXI H, 2000H      ; Address of number N
2      MOV A, M          ; A = N
3      CPI 00H           ; Check if N = 0
4      JNZ CALC          ;
5      MVI A, 01H        ; 0! = 1
6      JMP STORE         ;
7  CALC:
8      MOV B, A          ; B = N
9      DCR B             ; B = N - 1
10     FACT_LOOP:
11         MOV C, B       ; Multiplier
12         MOV D, A       ; Multiplicand
13         MVI A, 00H     ; Accumulator for addition
14     MUL_LOOP:
15         ADD D           ; A = A + D
16         DCR C           ; Decrement multiplier
17         JNZ MUL_LOOP   ; Repeat until C = 0
18
19         DCR B           ; Next factorial term (N-2, N-3...)
20         JNZ FACT_LOOP
21     STORE:
22         INX H
23         MOV M, A        ; Store 8-bit result at 2001H
24         HLT
```

Flowchart



4. Square Root of a Number

Logic: We subtract consecutive odd numbers (1, 3, 5, 7...) from the given number until the number becomes zero (or less than the next odd number). The number of subtractions performed is the integer square root.

```

1      LXI H, 2000H      ; Address of the number
2      MOV A, M          ; Load number into Accumulator
3      MVI B, 01H        ; B = First odd number (1)
4      MVI C, 00H        ; C = Square root counter
5
6  SQRT_LOOP:
7      CMP B             ; Compare A with odd number
8      JC DONE           ; If A < odd number, we are done
9
10     SUB B              ; Subtract odd number from A
11     INR C              ; Increment square root counter
12     INR B              ; B = B + 1
13     INR B              ; B = B + 1 (Next odd number)
14     JMP SQRT_LOOP     ; Repeat
15
16  DONE:
17     INX H              ; Increment address
18     MOV M, C           ; Store square root at 2001H
19     HLT

```

5. Convert 2-digit BCD to 8-bit Binary with Flowchart

Logic: A 2-digit BCD number consists of tens (upper nibble) and units (lower nibble). To convert to binary (hexadecimal), extract the tens digit, multiply it by 10 (0AH), and add the units digit.

Assembly Code

```

1      LXI H, 2000H      ; Address of BCD input
2      MOV A, M          ; Load BCD number
3      MOV B, A          ; Save a copy in B
4
5      ; Extract Tens (upper nibble)
6      ANI FOH           ; Mask lower nibble
7      RRC               ; Shift right 4 times
8      RRC
9      RRC
10     RRC
11     MOV C, A           ; C = Tens digit
12
13     ; Multiply Tens by 10 (0AH)
14     MVI D, 0AH         ; Multiplier = 10
15     MVI A, 00H         ; Accumulator for multiplication
16  MUL_10:
17     ADD C              ; Add tens digit
18     DCR D              ; Decrement multiplier
19     JNZ MUL_10
20
21     ; Extract Units and Add

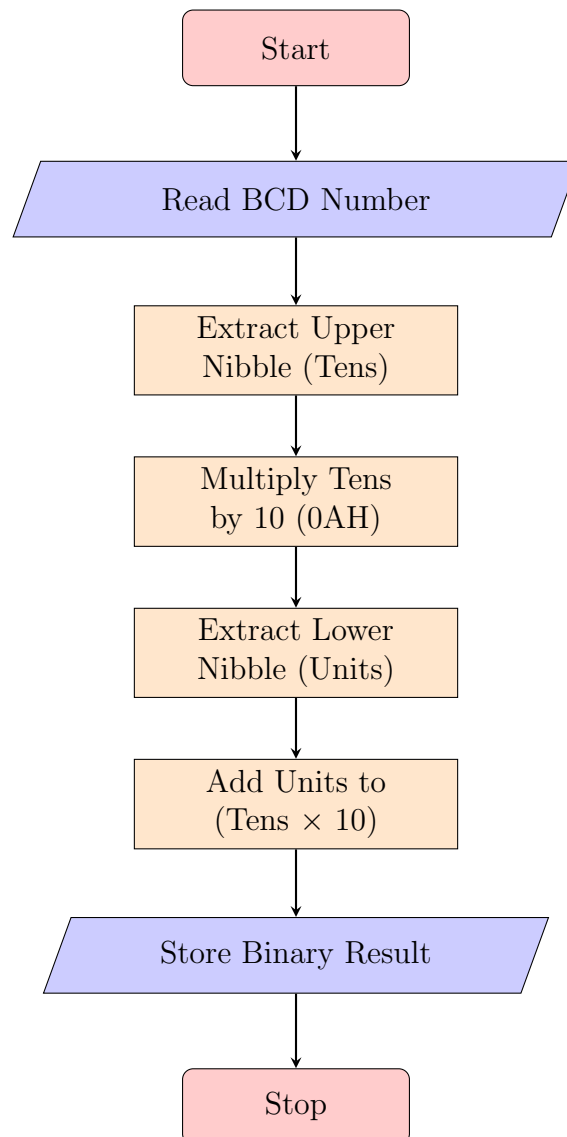
```

```

22  MOV C, A          ; C = (Tens * 10)
23  MOV A, B          ; Restore original BCD
24  ANI 0FH          ; Mask upper nibble (Units digit)
25  ADD C             ; A = Units + (Tens * 10)
26
27  INX H
28  MOV M, A          ; Store Binary result at 2001H
29  HLT

```

Flowchart



6. Generate 1 ms Pulse via I/O Port using RST 7.5

Logic: The main program must unmask the RST 7.5 interrupt and wait. When the button is pressed, the CPU vectors to '003CH'. The Interrupt Service Routine (ISR) will send a HIGH signal to Port 01H, run a delay loop for 1 ms, and then send a LOW signal to Port 01H to complete the pulse.

Assuming a standard 3 MHz clock frequency for the 8085, 1 ms requires 3000 T-states. A simple 8-bit loop ('DCR' + 'JNZ') takes 14 T-states. Count $\approx 3000/14 \approx 214_{10} = D6H$.

```

1 ; ===== MAIN PROGRAM =====
2     ORG 2000H           ; Main Program Start
3     MVI A, 0BH          ; Unmask RST 7.5 (0000 1011)
4     SIM                 ; Set Interrupt Mask
5     EI                  ; Enable Interrupts
6 WAIT:
7     HLT                 ; Wait for interrupt
8     JMP WAIT            ; Safety loop
9
10 ; ===== ISR FOR RST 7.5 =====
11     ORG 003CH           ; Vector address for RST 7.5
12
13     MVI A, 01H          ; Set LSB High
14     OUT 01H             ; Send HIGH to Port 01H to start pulse
15
16     ; Call 1 ms Delay
17     MVI C, D6H          ; Load calculated delay count
18 DELAY:
19     DCR C               ; 4 T-States
20     JNZ DELAY           ; 10/7 T-States (Total 14 T-states per loop)
21
22     MVI A, 00H          ; Set LSB Low
23     OUT 01H             ; Send LOW to Port 01H to end pulse
24
25     EI                  ; Re-enable interrupts
26     RET                 ; Return to main program

```

[12pt, a4paper]article

[margin=1in]geometry amsmath amssymb graphicx tabularx array [table]xcolor tikz listings

8085 Microprocessor Interfacing & Programming Jadavpur University - Electrical Engineering (2023)

1. Memory and I/O Interfacing Diagram (Partial Decoding)

Problem Statement: Draw a diagram for interfacing RAM of size 2K and I/O with 8085 CPU at addresses 2000H and 4000H for RAM, and 17H and 07H for I/O. Use partial memory and I/O address decoding method.

Address Decoding Logic

Memory Decoding (Partial)

A 2K RAM requires 11 address lines (A_{10} to A_0). We will use A_{15}, A_{14}, A_{13} to decode the Chip Select ($\overline{CS}$) and ignore A_{12}, A_{11} (Partial Decoding).

RAM 1 (2000H): Binary is 0010 0000 0000 0000. Here $A_{15} = 0, A_{14} = 0, A_{13} = 1$.

RAM 2 (4000H): Binary is 0100 0000 0000 0000. Here $A_{15} = 0, A_{14} = 1, A_{13} = 0$.

Using a 74LS138 (3-to-8 decoder) enabled by $\overline{IO/\overline{M}} = 0$, we can express the active-low chip select signals as:

$$\overline{CS_{RAM1}} = \overline{IO/\overline{M}} \cdot \overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{13}$$

$$\overline{CS_{RAM2}} = \overline{IO/\overline{M}} \cdot \overline{A_{15}} \cdot A_{14} \cdot \overline{A_{13}}$$

I/O Decoding (Partial)

8-bit I/O addresses use lines A_7 to A_0 . We can decode using the lower nibble and A_4 , ignoring A_7, A_6, A_5 .

I/O Device 1 (07H): Binary is 0000 0111. Decode $A_4 = 0, A_3 = 0, A_2 = 1, A_1 = 1, A_0 = 1$.

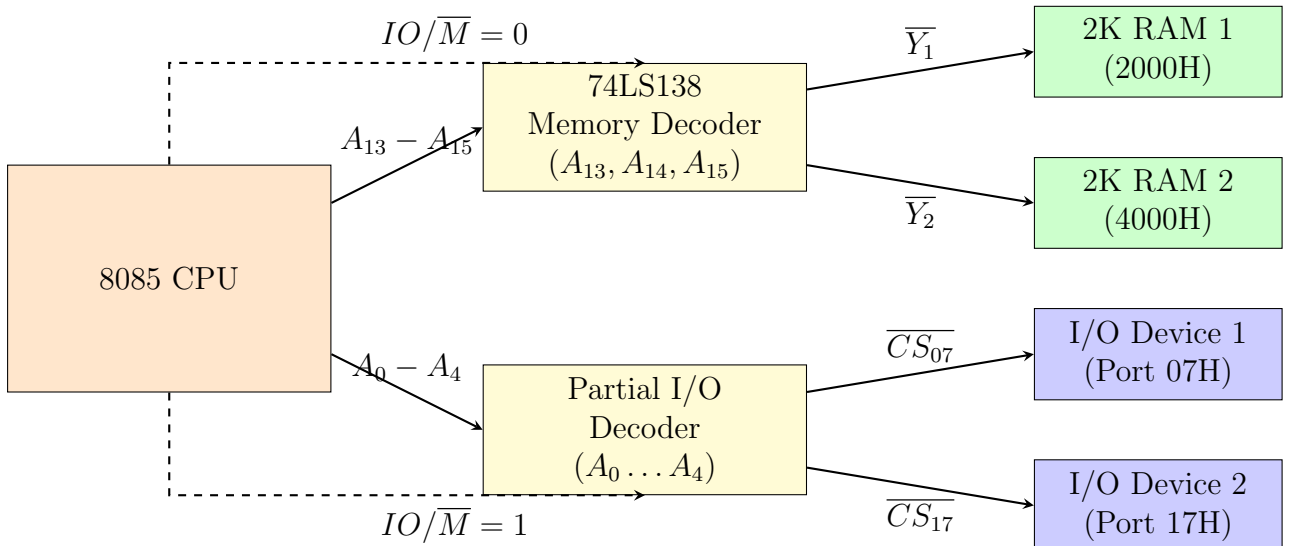
I/O Device 2 (17H): Binary is 0001 0111. Decode $A_4 = 1, A_3 = 0, A_2 = 1, A_1 = 1, A_0 = 1$.

The active-low chip select signals for I/O (enabled by $IO/\overline{M} = 1$) are:

$$\overline{CS_{07H}} = \overline{IO/\overline{M}} \cdot \overline{A_4} \cdot \overline{A_3} \cdot A_2 \cdot A_1 \cdot A_0$$

$$\overline{CS_{17H}} = \overline{IO/\overline{M}} \cdot A_4 \cdot \overline{A_3} \cdot A_2 \cdot A_1 \cdot A_0$$

Block Diagram Representation



2. Assembly Language Programs

i. Perform the same task as XTHL

Logic: The XTHL instruction exchanges the logical pair H and L with the top two bytes of the stack. Specifically, the contents of register L are exchanged with the stack location pointed to by SP, and register H is exchanged with SP+1. We can replicate this using standard PUSH and POP instructions alongside the DE register pair.

```
1      ; Replicate the behavior of XTHL
2      POP D          ; Pop top 2 bytes of stack into D and E
3                      ; E = (SP), D = (SP+1)
4
5      PUSH H          ; Push current H and L onto the stack
6                      ; (SP-1) = H, (SP-2) = L. SP returns to original
7                      ; position.
8
9      MOV H, D         ; Load H with old top of stack (SP+1)
10     MOV L, E         ; Load L with old top of stack (SP)
```

ii. Find the nth Fibonacci number, the value of n being 4

Logic: The Fibonacci sequence generates the next number by adding the two preceding ones. Assuming the sequence starts as 0, 1, 1, 2, 3, 5... We map $F(0) = 0$, $F(1) = 1$, $F(2) = 1$, $F(3) = 2$, and $F(4) = 3$. We will use registers to keep track of $F(n-2)$ and $F(n-1)$ and loop until n is reached.

```
1      LXI H, 2000H     ; Memory pointer to store the final result
2      MVI C, 04H       ; Counter for n = 4
3
4      MVI B, 00H       ; B = F(0) = 0 (Holds F(n-2))
5      MVI D, 01H       ; D = F(1) = 1 (Holds F(n-1))
6
7      ; Edge cases for n=0 or n=1
8      MOV A, C
9      CPI 00H
10     JZ STORE_0        ; If n=0, store 0
11     DCR C
12     JZ STORE_1        ; If n=1, store 1
13
14 FIB_LOOP:
15     MOV A, B           ; A = F(n-2)
16     ADD D              ; A = F(n-2) + F(n-1) -> This is F(n)
17
18     ; Shift values for next iteration
19     MOV B, D           ; Old F(n-1) becomes new F(n-2)
20     MOV D, A           ; New F(n) becomes new F(n-1)
21
22     DCR C              ; Decrement counter n
23     JNZ FIB_LOOP      ; Loop until C reaches 0
```

```

24      MOV M, A          ; Store the nth Fibonacci number
25      HLT
26
27
28 STORE_0:
29      MOV M, B          ; Store 0
30      HLT
31
32 STORE_1:
33      MOV M, D          ; Store 1
34      HLT

```

Alternative to ii. Convert Hexadecimal to Binary

Note: Hexadecimal and Binary are conceptually the same within the CPU. This standard exam question typically expects an ALP that takes an 8-bit Hex number and "unpacks" it into 8 contiguous memory locations, storing 01H or 00H representing the binary string, or converting an ASCII hex character to its binary value.

Logic (Unpacking Hex to Binary String): We rotate the given number left bit-by-bit. The Carry flag catches the bit, which we then store in an array.

```

1      LXI H, 2000H      ; Source address of the Hex number
2      MOV A, M          ; Load Hex number into Accumulator
3      LXI H, 2001H      ; Destination array start address
4      MVI C, 08H        ; Loop counter for 8 bits
5
6 BIN_LOOP:
7      RLC               ; Rotate Accumulator Left; MSB goes into
8      Carry             ; If Carry is 1, jump to store 1
9      JC STORE_ONE
10
11 STORE_ZERO:
12      MVI M, 00H        ; Store 00H in memory
13      JMP CONTINUE
14
15 STORE_ONE:
16      MVI M, 01H        ; Store 01H in memory
17
18 CONTINUE:
19      INX H              ; Move pointer to next memory location
20      DCR C              ; Decrement bit counter
21      JNZ BIN_LOOP      ; Repeat for all 8 bits
22      HLT

```